JHARKHAND ECO & CULTURAL TOURISM

COMPLETE BACKEND ENGINEERING & IMPLEMENTATION REPORT

Backend-first implementation specification | Version 1.0 | 15 October 2026

A development-ready technical blueprint covering platform scope, module boundaries, PostgreSQL schema inventory, REST APIs, JWT and internal-service authentication, transactional reservations, payments, Docker infrastructure, testing and production acceptance.

Architecture: Node.js (JavaScript) • Express modular monolith • PostgreSQL/PostGIS • Redis/BullMQ • Nginx • Docker Compose • future Next.js BFF

Executive overview

The objective is to deliver the entire Jharkhand eco and cultural tourism backend before developing the web frontend. The backend will serve a public discovery website, tourists, local guides, accommodation partners, package/experience operators, and platform administrators. It is one deployable Express modular monolith plus independent worker process, not a collection of HTTP microservices.

Architectural decision: users sign in using JWT access and refresh credentials; the later Next.js BFF authenticates itself to Express with a server-only static internal token, while Express independently validates user JWTs and roles for protected endpoints. This separation prevents a shared service secret from acting as universal user authorization.

All runtime dependencies - Node.js application, PostgreSQL/PostGIS, Redis, BullMQ worker, Nginx - will be containerized. The only required host runtime is Docker with Compose. Package installation occurs inside Docker images. A separately configured S3-compatible object store is required for uploaded media.

This report is a system specification, not generated application source code. Its inventories represent the agreed scope; detailed final DDL, individual field validation rules, selected payment gateway, email provider and deployment secrets are implementation decisions to confirm when implementing modules.

Specification item	Decision / count
Architecture	Express modular monolith + worker
Logical domain modules	22
Database table groups	13
Proposed tables	78
Listed REST routes	241
Infrastructure	Docker Compose + PostGIS + Redis + Nginx

How to use this document

1. Treat section 2 as the functional scope and exclusions; obtain sign-off before implementation.
2. Build the migrations and route contracts from sections 4-6, preserving module ownership.
3. Implement domain modules in the dependency order in section 12; complete each gate before advancing.
4. Use the final acceptance list to determine whether backend is truly ready for frontend development.

1. Objectives, system boundaries and users

Primary objective: promote Jharkhand nature and cultural heritage and let a tourist discover destinations, browse or reserve hotels/resorts, book guided services and cultural activities, select tour packages, create itineraries and pay online. Partners maintain approved offerings; administrators manage content, verification and commerce operations.

Persona	Capabilities	Boundary
Visitor	Browse public destinations, search, view properties, packages, guides, culture and CMS pages	Cannot create bookings without account
Tourist	Account/profile, favorites, trips, quotes, booking, payments, cancellation, reviews	Own records only
Guide	Maintain guide identity, services, verification, availability, eligible bookings	Owned organization/resource only
Accommodation partner	Property, room-type, inventory, rates, policies and booking management	Owned property only
Package/experience operator	Publish departure/sessions, run bookings, maintain package itinerary	Owned supply only
Admin / super admin	Verify partner supply, moderate CMS and reviews, handle bookings/refunds, reporting	Permission-gated, audited

1.1 Explicitly included

- Geography, district and tourism-category master data, rich attractions and nearby discovery.

- Bookable accommodation (room-type inventory), bookable guide slots, bookable cultural/nature experience sessions and tour-package departures.
- Transactional booking, optional approval/request-to-book, payment verification, cancellation/refund support, notifications and voucher handling.
- Dynamic CMS for homepage, navigation, announcements, page sections and media; partner and admin operations.
- Saved manual tourist itineraries, favorites, reviews, reports, security and audit logging.

1.2 Explicit exclusions from v1

Flights, railway tickets, long-distance bus inventory, taxi/fleet dispatch, insurance, AI itinerary generation/chatbot, social networking, marketplace wallet, full ERP/accounting, native mobile-only APIs, microservice mesh, Kubernetes, Elasticsearch and a separate analytics warehouse. These are not prerequisites for this stated scope.

1.3 Operational assumptions needing project confirmation

- One currency (INR) initially; configurable currency code in financial records. Tax calculation configuration and actual GST treatment require finance/legal sign-off.
- Availability types: accommodation by room type and stay date; guides by time slots; experiences and package departures by session/date capacity.
- Booking types can operate as instant-book or request-to-book; partner approval window and hold expiration must be defined.
- No automatic partner settlement/payout engine is assumed; if the platform collects funds on behalf of partners, settlement accounting and tax design must be specified before production payment launch.
- Login baseline: email/password with email verification; SMS OTP/social identity are not included by default.
- Uploaded images/documents use an S3-compatible object store and signed URLs, not database BLOB fields.

2. High-level technical architecture

```
Tourist / Partner / Admin browser
      | HTTPS
      v
Future Next.js App / BFF (server-side)
  - secure HttpOnly session cookies
  - user JWT forwarding on protected API calls
  - X-Internal-Service-Token injected server-side
      | private Docker network
      v
Nginx (private routing) --> Express API modular monolith
                                  |         |       \
                                  v         v        v
                            PostgreSQL/     Redis    Object Store
                              PostGIS       (cache + BullMQ)
                                            |
                                            v
                                  Independent BullMQ worker
```

External payment providers call a narrowly exposed webhook route that verifies the provider signature; they do not use the BFF shared service token. The diagram is a conceptual deployment shape: in production the public TLS termination and webhook ingress route must be configured explicitly.

2.1 Technology baseline

Layer	Choice	Reason / non-goal
Backend runtime	Node.js, plain JavaScript	Avoid mandatory TypeScript conversion
HTTP framework	Express.js, versioned REST	One API host with separated domain modules
Database	PostgreSQL with PostGIS	Transactions, constraints and nearby queries
Background jobs	Redis with BullMQ	Retries, expirations, email and media work
Proxy	Nginx	TLS routing, request limits and access

Layer	Choice	Reason / non-goal
		controls
Media	S3-compatible object storage	Private verification docs, managed public images
API definition	OpenAPI 3.x; schema validation	Contract-first frontend integration
Deployment	Docker Compose	No local Node/Postgres/Redis installation required

2.2 Code ownership and module discipline

One application serves /api/v1. Each module owns routing, controllers, input schemas, services, repositories, policies, and optional domain events. Controllers never implement booking/payment rules. Cross-module references use documented service interfaces or domain events; do not directly import foreign repositories. Administrative routes reuse the same domain services.

```
backend/src/
  app.js  server.js
  config/{env,database,redis,storage}.js
  core/{middleware,errors,events,logging,utils,database}/
  modules/{auth,users,geography,destinations,experiences,
           properties,guides,packages,trips,search,bookings,
           payments,partners,reviews,favorites,cms,media,
           notifications,administration,analytics,audit,settings}/
    <module>/{*.routes.js,*.controller.js,*.service.js,
              *.repository.js,*.schema.js,*.policy.js}
  jobs/{queues.js,worker.js,processors/}
  database/{migrations,seeds}/
backend/tests/{unit,integration,api}/
infrastructure/nginx/default.conf
Dockerfile  docker-compose.yml  .env.example  docs/{api,architecture,database,permissions}/
```

3. Backend domain services / functional modules

Module	Required capability
01 Core & Configuration	Environment checking; health checks; standard API errors; logging; input validation; database/Redis connections.
02 Authentication	Registration, login, JWT access/refresh, rotation and revocation, reset and verification flows.
03 Users & RBAC	User profile, roles, permission codes, partner ownership and delegated organization access.
04 Geography	Regions, Jharkhand districts, coordinates and geo reference data.
05 Destinations	Attractions, category links, media, FAQs, rich publishing and related places.
06 Experiences	Cultural and nature experiences, session capacity, operational publishing.
07 Accommodation	Properties, rooms, rates, policies, amenities, date-wise inventory and approvals.
08 Guides	Guide verification, languages, service offers, geography and time availability.
09 Packages	Package master data, linked attractions, detailed day plans, inclusions and dated departures.
10 Trip Planner	Private draft trip days and stops; no implicit commercial reservation.
11 Discovery/Search	Unified search, autocomplete, nearby results and filter metadata.
12 Bookings/Inventory	Quotes, holds, booking state transitions, cancellations and documents.
13 Payments/Refunds	Gateway order creation, verified events, idempotent reconciliation and refunds.
14 Reviews/Favorites	Reviews with verification semantics and saved item relationships.
15 CMS/Media	Approved section registry, navigation, landing pages, uploaded asset security.
16 Partner Management	Applications, memberships, documents and approval history.

Module	Required capability
17 Notifications	Email and in-app notification generation, delivery tracking and templates.
18 Administration	Moderation and operational views backed by existing module services.
19 Reporting	SQL-derived booking, payment, partner and destination performance.
20 Audit & Security	Admin audit events, login protections, resource access enforcement.
21 Background Workers	BullMQ queues, retries, expiry, outbox dispatch and operational recovery.
22 Platform Settings	Policy keys, transaction-related parameters and protected system settings.

4. PostgreSQL data model and table inventory

The following is the authoritative proposed table checklist. Counts are calculated from the actual entries below. UUIDs are standard primary keys unless an associative table uses a composite primary key or settings uses a key PK. Include created_at and updated_at on mutable business records, sensible foreign keys, strict CHECK constraints, indexed filter columns, and UTC timestamps.

Schema notation: “optional” or NULL indicates a nullable attribute; JSONB is only for strongly validated structured snapshots or CMS data. Amounts use NUMERIC(12,2), capacity uses INTEGER with nonnegative CHECK, timestamps use TIMESTAMPTZ. Administrative status changes are audited.

4.1 Identity, authentication and RBAC (9 tables)

Table	Core fields / relationships
users	id; full_name; email; phone; password_hash; status; email_verified_at; phone_verified_at; last_login_at
user_profiles	user_id PK/FK; avatar_media_id; preferred_language; country; state; city
roles	id; code UNIQUE; name; description
permissions	id; code UNIQUE; module; description
role_permissions	role_id; permission_id; composite PK
user_roles	user_id; role_id; composite PK
auth_refresh_tokens	id; user_id; token_hash; token_family_id; expires_at; revoked_at; replaced_by_id
auth_verification_tokens	id; user_id; token_hash; purpose; expires_at; consumed_at
auth_login_attempts	id; identifier_hash; ip_address; succeeded; attempted_at

4.2 Geography and destinations (8 tables)

Table	Core fields / relationships
regions	id; name; slug UNIQUE; description; status
districts	id; region_id; name; slug UNIQUE; description; location; status
destinations	id; district_id; name; slug UNIQUE; summary; description; geo_location; address; opening_hours JSONB; best_time_to_visit; entry_fee; status; featured
destination_categories	id; name; slug UNIQUE; description; icon
destination_category_links	destination_id; category_id; composite PK
destination_media	destination_id; media_id; sort_order; is_cover
destination_faqs	id; destination_id; question; answer; sort_order
destination_related_places	destination_id; related_destination_id; sort_order

4.3 Cultural and ecotourism experiences (3 tables)

Table	Core fields / relationships

Table	Core fields / relationships
experiences	id; partner_id; destination_id; name; slug UNIQUE; description; duration_minutes; min_participants; max_participants; status
experience_media	experience_id; media_id; sort_order
experience_sessions	id; experience_id; starts_at; ends_at; capacity; price_per_person; status

4.4 Accommodation / inventory (10 tables)

Table	Core fields / relationships
properties	id; partner_id; district_id; destination_id; name; slug UNIQUE; property_type; description; address; geo_location; contact_phone; check_in_time; check_out_time; status
property_media	property_id; media_id; sort_order; is_cover
amenities	id; code UNIQUE; name; category; icon
property_amenities	property_id; amenity_id; composite PK
room_types	id; property_id; name; description; max_adults; max_children; total_units; base_price; status
room_type_media	room_type_id; media_id; sort_order
room_type_amenities	room_type_id; amenity_id; composite PK
room_inventory	id; room_type_id; stay_date; total_units; blocked_units; UNIQUE(room_type_id,stay_date)
room_rates	id; room_type_id; stay_date; nightly_price; currency; UNIQUE(room_type_id,stay_date)
property_policies	id; property_id UNIQUE; cancellation_policy; child_policy; house_rules; identification_policy

Use room inventory at room-type/date grain. Compute sellable units considering active bookings and unexpired holds under transactions. Do not add physical room assignment unless the business actually needs individual room numbers.

4.5 Tour guides (7 tables)

Table	Core fields / relationships
guides	id; user_id; partner_id; bio; years_experience; verification_status; status
guide_languages	id; code UNIQUE; name
guide_language_links	guide_id; language_id; composite PK
guide_destinations	guide_id; destination_id; composite PK
guide_services	id; guide_id; title; duration_minutes; price; max_group_size; description; status
guide_availability	id; guide_id; service_date; start_time; end_time; status
guide_verification_documents	id; guide_id; document_media_id; document_type; review_status; reviewed_by

4.6 Tour packages (7 tables)

Table	Core fields / relationships
packages	id; partner_id; name; slug UNIQUE; summary; description; duration_days; duration_nights; starting_price; min_travelers; max_travelers; status
package_destinations	package_id; destination_id; sort_order
package_itinerary_days	id; package_id; day_number; title; description
package_itinerary_items	id; itinerary_day_id; destination_id; activity_title; description; sort_order
package_inclusions	id; package_id; description; inclusion_type; sort_order
package_media	package_id; media_id; sort_order
package_departures	id; package_id; start_date; end_date; total_seats; price_per_person; status

4.7 Tourist itinerary planning (3 tables)

Table	Core fields / relationships
trips	id; user_id; title; start_date; end_date; traveler_count; status
trip_days	id; trip_id; trip_date; day_number
trip_stops	id; trip_day_id; destination_id; title; notes; start_time; end_time; sort_order

4.8 Booking and inventory holds (7 tables)

Table	Core fields / relationships
bookings	id; booking_reference UNIQUE; user_id; booking_type; booking_status; payment_status; currency; subtotal; tax_amount; discount_amount; total_amount; confirmed_at; cancelled_at
booking_items	id; booking_id; item_type; item_reference_id; description_snapshot; quantity; unit_price; tax_amount; total_amount; booking_details JSONB
booking_travelers	id; booking_id; full_name; age_optional; is_primary
inventory_holds	id; booking_item_id; resource_type; resource_id; start_at; end_at; quantity; expires_at; status
booking_status_history	id; booking_id; old_status; new_status; changed_by; reason; created_at
booking_cancellations	id; booking_id; reason; cancellation_fee; refund_amount; requested_by; status; created_at
booking_documents	id; booking_id; media_id; document_type; created_at

booking_items and inventory_holds must not rely only on a loose polymorphic resource ID. The booking service must validate type and ownership, snapshot item terms, and enforce resource-specific capacity via locked rows/constraints. Explicit per-type bridge tables can be added if strict DB foreign keys become necessary; do not claim a polymorphic UUID has FK integrity.

4.9 Payments and refunds (4 tables)

Table	Core fields / relationships
payment_orders	id; booking_id; provider; provider_order_id; amount; currency; status; expires_at
payment_transactions	id; payment_order_id; provider_payment_id; amount; status; paid_at; failure_reason
payment_webhook_events	id; provider; provider_event_id; payload JSONB; processing_status; received_at; processed_at
refunds	id; booking_id; payment_transaction_id; provider_refund_id; amount; reason; status; requested_at; processed_at

Never store full payment card details. Store provider IDs, status, amount and minimal signed-webhook event payloads with redaction and retention policy.

4.10 Partners and verification (4 tables)

Table	Core fields / relationships
partners	id; partner_type; business_name; legal_name; email; phone; address; tax_identifier_optional; status
partner_users	partner_id; user_id; role; composite PK
partner_documents	id; partner_id; media_id; document_type; review_status; reviewed_by
partner_approval_history	id; partner_id; previous_status; new_status; reviewed_by; notes; created_at

4.11 Reviews and favorites (2 tables)

Table	Core fields / relationships

Table	Core fields / relationships
reviews	id; user_id; booking_id NULL; subject_type; subject_id; rating; title; comment; status; created_at
favorites	id; user_id; subject_type; subject_id; created_at

4.12 CMS and media (7 tables)

Table	Core fields / relationships
media_assets	id; uploaded_by; storage_key; public_url_optional; mime_type; file_size; width; height; alt_text; visibility; status
cms_pages	id; title; slug; page_type; seo_title; seo_description; status; published_at
cms_page_sections	id; page_id; section_type; section_data JSONB; sort_order; is_enabled
cms_menus	id; code; name; status
cms_menu_items	id; menu_id; parent_id; label; link_type; link_value; sort_order; is_enabled
cms_redirects	id; source_path; target_path; status_code
cms_announcements	id; title; message; link; starts_at; ends_at; status

Uploads for private verification documents must remain access-controlled; public tourism images may use public read/CDN URLs. CMS JSON follows allowlisted section definitions.

4.13 Notifications, audit and operations (7 tables)

Table	Core fields / relationships
notification_templates	id; code; channel; subject; body; status
notifications	id; user_id; type; title; message; related_type; related_id; read_at; created_at
notification_deliveries	id; notification_id_optional; recipient; channel; provider_message_id; status; attempts; last_error
platform_settings	key PK; value_json; updated_by; updated_at
audit_logs	id; actor_user_id; action; entity_type; entity_id; old_values; new_values; ip_address; created_at
outbox_events	id; event_type; aggregate_type; aggregate_id; payload; status; created_at; processed_at
idempotency_records	id; scope; idempotency_key; request_hash; response_status; response_body; expires_at

4.14 Required database constraints and indexing

Area	Constraint / index acceptance rule
Identity	UNIQUE normalized user email; unique phone when provided; hashed refresh/verification tokens; non-expired/revoked checks.
Geography	Unique district/region slugs; valid latitude/longitude; GiST spatial indexes for PostGIS columns.
Content	Unique published slugs; section sort order per page; referential integrity for media relations.
Hotels	UNIQUE(room_type_id,stay_date) on inventory/rates; CHECK(blocked_units>=0 AND total_units>=blocked_units).
Guides	Prevent conflicting committed/held guide times; PostgreSQL exclusion constraints or resource-level transactional locks.
Sessions/departures	No confirmed+held allocation greater than capacity; deterministic transactional locking.
Bookings	Unique reference; state transition allowlist; price/currency immutable once order is created except explicit documented revisions.
Payments	UNIQUE(provider,provider_order_id), UNIQUE(provider,provider_payment_id), UNIQUE(provider,provider_event_id); idempotency records.

Area	Constraint / index acceptance rule
Reviews & favorites	Unique(user_id,subject_type,subject_id) favorites; one eligible verified review per booking+subject.
Operational	Indexes on status, date, owner/partner, booking reference, payment IDs and expiration scan fields.

4.15 Referential integrity and deletion policy

- References from bookings and payment records must not cascade-delete operational history. Use RESTRICT, historical snapshots, or soft archive on supply resources.
- Partnership membership is distinct from global role membership. Every partner query filters by approved organization and resource ownership.
- Published destinations may be archived, but linked commercial bookings, reviews and audit records remain consistent.
- Create migration scripts and rollback/repair plans for all tables; initial Jharkhand district/category seeds should be version controlled.
- Do not store precise identity-document files or sensitive payment credentials in database text fields; store restricted object keys with access policy.

5. REST API specification / endpoint register

All endpoints are mounted under /api/v1 except operational health paths, which may be exposed separately. /webhooks/payments/:provider is a special public ingress route with provider-signature verification and IP/rate controls; it does not accept the shared BFF credential. Public GET listing endpoints return published/approved inventory only.

Auth notation: P = public read via trusted BFF service token; U = authenticated user JWT + BFF service token; PARTNER = valid user+partner membership+ownership; ADMIN = required admin permission; WEBHOOK = verified provider signature. Route grouping indicates the baseline policy; exact operation-to-permission rules are documented in section 7.

5.1 Authentication, user profile and RBAC (21 routes)

Verb	Endpoint (prefix /api/v1)	Behavior
POST	/auth/register	Create user account
POST	/auth/login	Authenticate and issue access/refresh tokens
POST	/auth/refresh	Rotate refresh token
POST	/auth/logout	Revoke current refresh session
POST	/auth/logout-all	Revoke all sessions
GET	/auth/me	Return authenticated user and permissions
POST	/auth/forgot-password	Request password reset
POST	/auth/reset-password	Reset password with verification token
POST	/auth/change-password	Change password while logged in
POST	/auth/verify-email	Complete email verification
POST	/auth/resend-verification	Resend verification
GET	/users/me	Read personal profile
PATCH	/users/me	Update personal profile
GET	/users/me/roles	Read assigned roles
GET	/admin/users	Search user accounts
GET	/admin/users/:id	Read user account
PATCH	/admin/users/:id/status	Suspend or reactivate user
PUT	/admin/users/:id/roles	Set user roles
GET	/admin/roles	List roles
GET	/admin/permissions	List permissions
PUT	/admin/roles/:id/permissions	Set permissions for role

5.2 Geography and destinations (36 routes)

Verb	Endpoint (prefix /api/v1)	Behavior
GET	/regions	List regions

Verb	Endpoint (prefix /api/v1)	Behavior
GET	/districts	List districts
GET	/districts/:slug	Get district
GET	/districts/:slug/destinations	List district attractions
GET	/destination-categories	List destination categories
GET	/destinations	List/filter published destinations
GET	/destinations/featured	Get featured destinations
GET	/destinations/:slug	Get destination detail
GET	/destinations/:slug/nearby	Get nearby destinations
GET	/destinations/:slug/experiences	Get related experiences
GET	/destinations/:slug/properties	Get nearby properties
GET	/destinations/:slug/guides	Find guides for destination
GET	/destinations/:slug/reviews	Get destination reviews
GET	/admin/destinations	List destinations including drafts
POST	/admin/destinations	Create destination
GET	/admin/destinations/:id	Read admin destination detail
PATCH	/admin/destinations/:id	Update destination
PATCH	/admin/destinations/:id/status	Publish/unpublish destination
DELETE	/admin/destinations/:id	Archive destination
PUT	/admin/destinations/:id/categories	Replace destination categories
POST	/admin/destinations/:id/media	Attach destination media
DELETE	/admin/destinations/:id/media/:mediaId	Detach media
PUT	/admin/destinations/:id/faqs	Replace FAQs
PUT	/admin/destinations/:id/related	Set related destinations
GET	/admin/regions	List managed regions
POST	/admin/regions	Create region
PATCH	/admin/regions/:id	Update region
PATCH	/admin/regions/:id/status	Set region status
GET	/admin/districts	List managed districts
POST	/admin/districts	Create district
PATCH	/admin/districts/:id	Update district
PATCH	/admin/districts/:id/status	Set district status
GET	/admin/destination-categories	List managed categories
POST	/admin/destination-categories	Create category
PATCH	/admin/destination-categories/:id	Update category
PATCH	/admin/destination-categories/:id/status	Set category status

5.3 Bookable experiences (14 routes)

Verb	Endpoint (prefix /api/v1)	Behavior
GET	/experiences	List experiences
GET	/experiences/:slug	Read experience
GET	/experiences/:id/sessions	List sessions
GET	/experiences/:id/availability	Check seats
GET	/partners/me/experiences	List partner experiences
POST	/partners/me/experiences	Create experience
PATCH	/partners/me/experiences/:id	Edit experience
POST	/partners/me/experiences/:id/sessions	Create session
PATCH	/partners/me/experiences/:id/sessions/:sessionId	Update session
DELETE	/partners/me/experiences/:id/sessions/:sessionId	Remove unused session
POST	/partners/me/experiences/:id/media	Attach media
DELETE	/partners/me/experiences/:id/media/:mediaId	Detach media
GET	/admin/experiences	Moderation listing
PATCH	/admin/experiences/:id/status	Approve, publish or reject

5.4 Accommodation (26 routes)

Verb	Endpoint (prefix /api/v1)	Behavior
GET	/properties	Search accommodation
GET	/properties/:slug	Read property
GET	/properties/:id/rooms	List room types

Verb	Endpoint (prefix /api/v1)	Behavior
GET	/properties/:id/availability	Check dates, guests and prices
GET	/amenities	List amenities
GET	/partners/me/properties	List owned properties
POST	/partners/me/properties	Create property
PATCH	/partners/me/properties/:id	Edit property
PUT	/partners/me/properties/:id/amenities	Assign property amenities
PUT	/partners/me/properties/:id/policies	Edit property policies
POST	/partners/me/properties/:id/media	Attach images
DELETE	/partners/me/properties/:id/media/:mediaId	Detach image
POST	/partners/me/properties/:id/rooms	Create room type
PATCH	/partners/me/properties/:id/rooms/:roomId	Edit room type
PATCH	/partners/me/properties/:id/rooms/:roomId/status	Change room status
POST	/partners/me/properties/:id/rooms/:roomId/media	Attach room image
DELETE	/partners/me/properties/:id/rooms/:roomId/media/:mediaId	Detach room image
PUT	/partners/me/rooms/:roomId/inventory	Write inventory calendar
PUT	/partners/me/rooms/:roomId/rates	Write date-based rate calendar
GET	/partners/me/rooms/:roomId/calendar	Read calendar
GET	/admin/properties	Approval listing
GET	/admin/properties/:id	Property approval detail
PATCH	/admin/properties/:id/status	Approve/reject/suspend property
GET	/admin/amenities	Admin amenity list
POST	/admin/amenities	Add amenity
PATCH	/admin/amenities/:id	Edit amenity

5.5 Tour guides (22 routes)

Verb	Endpoint (prefix /api/v1)	Behavior
GET	/guides	Search guides
GET	/guides/:id	Read guide
GET	/guides/:id/services	Read guide services
GET	/guides/:id/availability	Read available slots
GET	/languages	List supported languages
GET	/partners/me/guides	List partner guides
POST	/partners/me/guides	Create guide profile
PATCH	/partners/me/guides/:id	Edit guide profile
PUT	/partners/me/guides/:id/languages	Set guide languages
PUT	/partners/me/guides/:id/destinations	Set guide destinations
POST	/partners/me/guides/:id/services	Create guide service
PATCH	/partners/me/guides/:id/services/:serviceId	Edit guide service
PATCH	/partners/me/guides/:id/services/:serviceId/status	Set guide service status
PUT	/partners/me/guides/:id/availability	Update guide availability
POST	/partners/me/guides/:id/documents	Submit verification documents
GET	/admin/guides	Verification/moderation list
GET	/admin/guides/:id	Guide review detail
PATCH	/admin/guides/:id/verification	Approve/reject verification
PATCH	/admin/guides/:id/status	Activate/suspend guide
GET	/admin/languages	Manage language list
POST	/admin/languages	Add supported language
PATCH	/admin/languages/:id	Update language

5.6 Tour packages (16 routes)

Verb	Endpoint (prefix /api/v1)	Behavior
GET	/packages	Browse packages
GET	/packages/:slug	Read package
GET	/packages/:id/departures	List package departures
GET	/packages/:id/availability	Get departure capacity
GET	/partners/me/packages	List owned packages
POST	/partners/me/packages	Create package
PATCH	/partners/me/packages/:id	Edit package

Verb	Endpoint (prefix /api/v1)	Behavior
PUT	/partners/me/packages/:id/destinations	Set linked destinations
PUT	/partners/me/packages/:id/itinerary	Save ordered days and activities
PUT	/partners/me/packages/:id/inclusions	Save included/excluded items
POST	/partners/me/packages/:id/media	Attach gallery
DELETE	/partners/me/packages/:id/media/:mediaId	Detach image
POST	/partners/me/packages/:id/departures	Create departure
PATCH	/partners/me/packages/:id/departures/:departureId	Edit departure
GET	/admin/packages	Moderation listing
PATCH	/admin/packages/:id/status	Approve/publish/reject

5.7 Saved trip planner (12 routes)

Verb	Endpoint (prefix /api/v1)	Behavior
GET	/trips	List own trips
POST	/trips	Create trip
GET	/trips/:id	Read trip
PATCH	/trips/:id	Edit trip
DELETE	/trips/:id	Delete draft trip
POST	/trips/:id/days	Add itinerary day
PATCH	/trips/:id/days/:dayId	Edit itinerary day
DELETE	/trips/:id/days/:dayId	Remove day
POST	/trips/:id/days/:dayId/stops	Add day stop
PATCH	/trips/:id/stops/:stopId	Edit stop
DELETE	/trips/:id/stops/:stopId	Delete stop
PATCH	/trips/:id/stops/reorder	Reorder stops

5.8 Search and discovery (4 routes)

Verb	Endpoint (prefix /api/v1)	Behavior
GET	/search	Unified search
GET	/search/suggestions	Autocomplete
GET	/search/nearby	Radius search using coordinates
GET	/search/filters	Available filter options

5.9 Bookings and inventory operations (16 routes)

Verb	Endpoint (prefix /api/v1)	Behavior
POST	/bookings/quote	Recompute authoritative price and availability
POST	/bookings	Create booking or approval request
GET	/bookings	List own bookings
GET	/bookings/:id	Get own booking detail
GET	/bookings/:id/status	Read current booking/payment state
POST	/bookings/:id/cancel	Request/perform cancellation
GET	/bookings/:id/documents	Retrieve permitted booking documents
POST	/bookings/:id/payment-order	Initiate payment once eligible
GET	/partners/me/bookings	Bookings for partner-owned supply
GET	/partners/me/bookings/:id	Get partner booking detail
POST	/partners/me/bookings/:id/approve	Approve request-to-book
POST	/partners/me/bookings/:id/reject	Reject request-to-book
GET	/admin/bookings	Search all bookings
GET	/admin/bookings/:id	View full booking
POST	/admin/bookings/:id/cancel	Authorized cancellation
POST	/admin/bookings/:id/refund	Authorized refund action

5.10 Payments and refunds (8 routes)

Verb	Endpoint (prefix /api/v1)	Behavior

Verb	Endpoint (prefix /api/v1)	Behavior
GET	/payments/orders/:id	Read payment order
GET	/payments/booking/:bookingId	Read booking payment history
POST	/payments/verify	Check gateway transaction signature/status
POST	/webhooks/payments/:provider	Receive gateway-verified webhook
GET	/refunds/:id	Read refund status
GET	/admin/payments	Search platform payments
GET	/admin/refunds	Search refunds
POST	/admin/refunds/:id/retry	Retry eligible refund

5.11 Partner onboarding (12 routes)

Verb	Endpoint (prefix /api/v1)	Behavior
POST	/partners/apply	Submit partner application
GET	/partners/me	Read own partner organization
PATCH	/partners/me	Update partner organization
POST	/partners/me/documents	Upload verification document
GET	/partners/me/users	List team members
POST	/partners/me/users	Add verified existing user
PATCH	/partners/me/users/:userId	Change partner-level role
DELETE	/partners/me/users/:userId	Remove organization membership
GET	/admin/partners	Review partner applications
GET	/admin/partners/:id	View partner detail
PATCH	/admin/partners/:id/status	Approve/reject/suspend partner
GET	/admin/partners/:id/documents	View verification documents

5.12 Reviews and favorites (9 routes)

Verb	Endpoint (prefix /api/v1)	Behavior
GET	/reviews	List public reviews for subject
POST	/reviews	Create review
PATCH	/reviews/:id	Edit own review
DELETE	/reviews/:id	Remove own review
GET	/favorites	List personal favorites
POST	/favorites	Save subject
DELETE	/favorites/:type/:id	Unsave subject
GET	/admin/reviews	List moderation queue
PATCH	/admin/reviews/:id/status	Approve/hide/reject

5.13 CMS and media (25 routes)

Verb	Endpoint (prefix /api/v1)	Behavior
GET	/content/pages/:slug	Read published public page
GET	/content/homepage	Read approved homepage sections
GET	/content/menus/:code	Read navigation menu
GET	/content/announcements	Read active notices
GET	/admin/cms/pages	List all pages
POST	/admin/cms/pages	Create page
GET	/admin/cms/pages/:id	Read page for editor
PATCH	/admin/cms/pages/:id	Edit page metadata
PUT	/admin/cms/pages/:id/sections	Replace ordered section configuration
PATCH	/admin/cms/pages/:id/status	Draft/publish/archive
DELETE	/admin/cms/pages/:id	Archive page
GET	/admin/cms/menus	Manage menus
POST	/admin/cms/menus	Create menu
PUT	/admin/cms/menus/:id/items	Replace ordered menu tree
GET	/admin/cms/redirects	Read redirects
POST	/admin/cms/redirects	Create redirect
DELETE	/admin/cms/redirects/:id	Delete redirect
GET	/admin/cms/announcements	Read announcements

Verb	Endpoint (prefix /api/v1)	Behavior
POST	/admin/cms/announcements	Create announcement
PATCH	/admin/cms/announcements/:id	Edit announcement
DELETE	/admin/cms/announcements/:id	Remove announcement
POST	/media/upload-init	Authorize signed media upload
POST	/media/upload-complete	Verify/register object metadata
GET	/media/:id	Read metadata or signed file URL
DELETE	/media/:id	Delete unreferenced allowed media

5.14 Notifications (6 routes)

Verb	Endpoint (prefix /api/v1)	Behavior
GET	/notifications	Read in-app inbox
GET	/notifications/unread-count	Get unread count
PATCH	/notifications/:id/read	Mark own item read
POST	/notifications/read-all	Mark all own items read
GET	/admin/notification-templates	List templates
PATCH	/admin/notification-templates/:id	Edit template

5.15 Administration, settings and reporting (14 routes)

Verb	Endpoint (prefix /api/v1)	Behavior
GET	/admin/dashboard	Admin KPIs
GET	/admin/reports/bookings	Booking conversion/performance
GET	/admin/reports/revenue	Paid/refunded totals
GET	/admin/reports/destinations	Destination booking performance
GET	/admin/reports/partners	Partner supply performance
GET	/partners/me/dashboard	Partner KPIs
GET	/partners/me/reports/bookings	Partner booking report
GET	/partners/me/reports/revenue	Partner revenue report
GET	/admin/settings	Read approved settings
PATCH	/admin/settings	Edit approved settings only
GET	/admin/audit-logs	Audit search
GET	/health/live	Process health (outside version prefix allowed)
GET	/health/ready	Dependency readiness (restricted)
GET	/health/version	Deployment information

5.16 API design conventions

- Use HTTP 200/201/202/204 intentionally; validation 400/422; authentication 401; insufficient permission 403; unknown 404; conflict/inventory 409; idempotency conflict 409; rate limit 429; server 5xx.
- Common response success: {success:true,data:{...},meta?:{page,limit,total}}. Common error: {success:false,error:{code,message,details?,requestId}}.
- Pagination: page>=1, limit bounded (for example maximum 100); stable sort and filter allowlists. Dates in ISO 8601, currency explicit, amounts decimal-safe.
- Do not permit arbitrary PATCH to payment or booking states. Expose narrow domain actions with audited state transitions.
- Make /bookings, payment creation, and refunds accept Idempotency-Key, bound to the user, scope, and request hash.
- Create OpenAPI schemas for each route: example requests/responses, validation limits, status codes, access policy, idempotency and side effects.
- The planned BFF may aggregate backend APIs but must not duplicate domain authorization, inventory calculation or payment verification.

5.17 Example API exchange (illustrative contract)

```
POST /api/v1/bookings/quote
X-Internal-Service-Token: (server-side secret)
Authorization: Bearer (user JWT)
Content-Type: application/json

{
  "itemType": "property_room",
  "resourceId": "<room-type-uuid>",
  "checkIn": "2026-12-10",
  "checkOut": "2026-12-12",
  "rooms": 1,
  "adults": 2,
  "children": 0
}

200 OK
{
  "success": true,
  "data": {
    "currency": "INR", "subtotal": "5600.00",
    "taxAmount": "0.00", "totalAmount": "5600.00",
    "available": true, "priceValidUntil": "<ISO timestamp>"
  }
}
```

The amounts and dates above illustrate format only; the real tax, availability and pricing are always computed on the server. A quote is not a reservation. Availability may change before an atomic booking hold is committed.

6. Identity, static service token and security model

6.1 Login and session lifecycle

5. BFF sends login credentials to Express with X-Internal-Service-Token; the user has no JWT on this request.
6. Express validates credentials; verifies status and login controls; issues short-lived access JWT and rotating opaque refresh token (or JWT refresh design with mandatory revocation storage).
7. BFF stores credentials in secure, HttpOnly, SameSite cookies with production Secure flag; avoid browser localStorage for access tokens.
8. Every subsequent BFF-to-Express protected request includes the static service token AND user access JWT. Express verifies signing, issuer/audience, expiration, permissions and resource ownership.
9. On expiry, BFF calls /auth/refresh, rotates refresh credential and updates cookies; logout invalidates refresh family/session.
10. Partner operations additionally validate approved partner organization membership and ownership, not just global user role.

6.2 Trust boundaries

Caller	Auth method	Required rule
Public browser	HTTPS to Next.js (later)	Never sees static token or backend private URL
BFF to private API	X-Internal-Service-Token	Secret comparison with constant-time semantics; network allowlist; rotate
Protected BFF to API	Service token + user JWT	Apply authorization + row/resource ownership on API
Payment webhook	Provider signature over raw body	Provider replay protection, event ID unique, validated provider/config
BullMQ worker	Private network / runtime secrets	Direct internal service interfaces; no external worker HTTP exposure

Store the static token only in server-side runtime configuration (.env outside version control for development, Docker secrets / secret manager when practical in production). Do not use NEXT_PUBLIC_ prefix. Shared internal token compromise is high impact; use rotation strategy and separate secrets per environment.

6.3 Required protections

- Use Argon2id or scrypt/bcrypt password hashing with measured security parameters; never store plaintext or reversible passwords.
- JWT claims include subject, issuer, audience, iat, exp, session/token identifier; validate accepted algorithm and keys.
- Limit login, reset, registration, payment and upload endpoints; use Redis-backed rate limiting and avoid account enumeration.
- Validate all request bodies, query parameters and filenames; prevent SSRF and path traversal for upload and external references.
- Use CSRF protection on cookie-authenticated BFF mutation endpoints; enforce SameSite/Origin where applicable.
- Use prepared queries/ORM parameterization; restrict DB credentials; do not log authorization headers, payment credentials or identity documents.
- CORS only permits intended frontend origin(s); do not mistake CORS for authentication.
- Log sensitive admin changes and payment/booking transitions with correlation IDs; define retention and access rights.

7. Roles, authorization and ownership matrix

Role	Allowed families	Enforcement
Tourist	browse published data; /users/me; /trips; own favorites; reviews; own bookings/payments	No other user or partner records
Guide	manage assigned guide profile, services, slots, documents; inspect entitled bookings	Must belong to guide organization/record
Hotel partner	manage owned property, room inventory and rates, view related reservations	Cannot alter others' properties
Package provider	manage owned packages/experiences and related reservations	Cannot approve own verification
Admin	moderation, CMS publishing, operational bookings/refunds subject to permission codes	Financial actions require elevated permission
Super admin	RBAC management, platform settings, administrative override with audit	No bypass of payment integrity

Seed permission codes by module and operation, e.g. destinations.read, destinations.create, destinations.publish, properties.manage.own, bookings.read.own, bookings.manage.platform, refunds.initiate, cms.publish, roles.assign, settings.write. Permissions are verified server-side per route; organization ownership is a separate mandatory predicate.

8. Commercial workflows and transaction safety

8.1 Instant hotel booking

11. Tourist requests quote. Service computes room-night subtotal and applicable fees/taxes from authoritative rates.
12. POST /bookings validates date range, guest capacity, room status and partner approval; transaction locks relevant room_inventory dates and checks allocations.
13. Create booking, booking_item snapshots and expiring inventory_holds, then commit. Return booking reference and expiry.
14. Payment order is generated for eligible held booking using a unique idempotency key; redirect/SDK can proceed.
15. On signed payment confirmation, persist webhook ID, validate amount/currency/order ownership and atomically transition held booking to confirmed; emit outbox event.
16. Worker generates confirmation/voucher and notifications; if payment fails/expires, release inventory under controlled state transition.

8.2 Guide, experience and package reservations

- Guide: time slot/time range must not overlap another held or confirmed guide reservation. Use transaction-level locking or exclusion constraints to enforce.
- Experience: lock session row and ensure remaining capacity >= requested participants; all quantities are counted once.
- Package departure: lock departure row and ensure seats available. Departure date changes cannot silently alter existing reservations.

- Request-to-book: create approval request with status; partner approval reserves eligible capacity and sets a payment deadline; rejection leaves no payable reservation.
- Cancellation policy: compute fees/refunds from the version of policy recorded with the booking, not from a policy edited after purchase.

8.3 Payment, refund and webhook state machine

```
Booking:   draft/requested -> pending_payment -> confirmed -> completed
                  |              |                 |
                  +-> rejected    +-> expired       +-> cancelled
                                 +-> cancelled
Payment:   created -> pending -> paid | failed | expired
Refund:    requested -> processing -> succeeded | failed
```

Actual allowed transitions must be documented in code and tests; the diagram is conceptual, not permission to jump between states. Payment failure and booking cancel do not necessarily imply an immediate refund; determine whether money was captured. Persist gateway webhook receipts before processing. Network retries must not duplicate a booking, payment order or refund.

8.4 Financial business-rule decisions

- Select gateway and define order/verification/refund semantics (the API abstraction remains gateway neutral).
- Decide whether platform is merchant of record or collects funds on behalf of partners; this determines legal/tax/settlement requirements.
- Define GST/taxes, cancellation/refund policy, booking fee treatment and receipt/invoice issuer with qualified finance advisers.
- If partner commissions or payouts are required, add a separate specified ledger/settlement module before accepting production partner funds; it is intentionally excluded here.

9. Redis, BullMQ, events and jobs

Queue	Trigger	Processor contract
booking-expiration	Inventory hold created	Recheck authoritative status; expire and release safely
booking-confirmation	Booking confirmed	Voucher, confirmation artifact, notifications
payment-reconciliation	Gateway state unclear	Query provider; idempotently reconcile
refund-processing	Refund authorized	Issue/retry gateway refund without duplicate refund
notification-delivery	Committed notification event	Email / in-app delivery record and retries
media-processing	Object upload completed	Validate/resize images and set metadata
outbox-dispatch	Outbox event committed	Reliably publish domain events to queues

BullMQ may be executed in the same Docker image as Express but with a separate worker command/container. Redis is not the authority for inventory or payments. Every processor has bounded retries, exponential backoff, explicit dead-letter/failed visibility, request ID correlation, deduplication and restart recovery.

10. Docker-only developer and deployment environment

The workstation/server needs Docker Engine plus Compose (or Docker Desktop). No direct npm install, PostgreSQL setup, PostGIS installation, Redis install or BullMQ daemon install on host. The build downloads packages inside Docker image layers using a committed package-lock.json.

10.1 Required services and data persistence

Compose service	Image/process	Network/volume
api	Built from backend/Dockerfile; Node/Express	Private app network; port 4000 internal
worker	Same image; worker.js entrypoint	Private app network
postgres	Pinned PostgreSQL+PostGIS image	postgres_data named volume; backups required

Compose service	Image/process	Network/volume
redis	Pinned Redis image with AOF if used	redis_data named volume; memory policy tuned
nginx	Pinned Nginx image	Public 80/443; route public BFF and webhook when added
object storage	Configured S3-compatible external endpoint or optional local service	Object durability and signed access

10.2 Required environment keys

Category	Environment keys / provision
Server	NODE_ENV, PORT, PUBLIC_BASE_URL, CORS_ORIGIN
Database	POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB; derived internal DATABASE_URL
Queue	REDIS_URL and worker job settings
Internal service	INTERNAL_SERVICE_TOKEN (separate by environment; never exposed to browser)
JWT	JWT_ACCESS_SECRET / public-private key configuration; JWT_REFRESH_SECRET, expirations, issuer/audience
Storage	STORAGE_ENDPOINT, BUCKET, ACCESS_KEY, SECRET_KEY, region/URL policy
Payments	PAYMENT_PROVIDER, PAYMENT_KEY_ID, PAYMENT_KEY_SECRET, PAYMENT_WEBHOOK_SECRET
Email	SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD, EMAIL_FROM
Observability	LOG_LEVEL, correlation ID, optional error collection config

10.3 Starter Docker Compose deployment contract

```
services:
  api:       # image built from ./backend; node src/server.js
  worker:    # same image; node src/jobs/worker.js
  postgres:  # pin PostGIS image; healthcheck; named volume
  redis:     # pin image; AOF optional; named volume
  nginx:     # proxy to api / future Next.js BFF; expose 80/443
volumes:
  postgres_data:
  redis_data:
networks:
  internal:
```

Use explicit health checks for database and Redis; separate migration job rather than running schema migrations unconditionally on every API start. Do not bind PostgreSQL/Redis to the public host. In production, set resource limits, log rotation, TLS, secret management, restart policy, backups, version pinning and restore drills. The default Nginx routing must not expose protected admin or backend internal endpoints directly.

10.4 Host commands (Docker only)

```
docker compose build
docker compose up -d
docker compose ps
docker compose logs -f api
docker compose logs -f worker
docker compose exec api node --version
# After scripts are added to package.json:
docker compose run --rm api npm run db:migrate
docker compose run --rm api npm run db:seed
docker compose run --rm api npm test
```

Command scripts such as db:migrate/db:seed/npm test are implementation requirements, not supplied application code. Do not run docker compose down -v against real data; that removes the named database volume.

11. API, database and application quality standards

Category	Definition of done
Contracts	OpenAPI 3.x for every endpoint, examples, permission map, error responses and stable pagination.
Validation	Strict request schema, cross-field booking date validation and normalized identity inputs.
Data	Migrations reviewed; referential integrity; transaction boundaries and indexes verified.
Monitoring	Liveness and readiness; structured logging; latency/error dashboards; queue failures and payment anomalies.
Security	Dependency scanning; secret rotation; rate limits; CSRF for BFF cookie routes; provider signatures.
Testing	Unit + DB integration + API + booking concurrency + gateway webhook replay and fault tests.
Recovery	Backups, restoration checks, restart resilience, reconciliation and orphan hold cleanup.
Performance	Query pagination; spatial indexes; bounded Redis cache TTL; load tests for search and booking.

11.1 Critical end-to-end test scenarios

- Register > verify email > login > refresh > logout; revoked refresh cannot be reused.
- Anonymous/unauthorized requests cannot access private profile, partner data or administration.
- Two simultaneous customers attempt last hotel room; exactly one reservation obtains capacity.
- Two users select same guide slot; only eligible non-overlapping reservations commit.
- Exactly-once effective payment processing under duplicate webhooks and repeated user callbacks.
- Payment arrives after inventory hold expires; system reconciles to a documented state, no hidden overbooking.
- Refund failure, retry and duplicate callback do not cause multiple refunds.
- Partner cannot change another partner's inventory or modify booking financial fields.
- CMS draft is not visible publicly; published safe component section is serialized correctly.
- Worker/API restart does not lose committed booking events; processing resumes from persisted outbox.
- Database backup can be restored with transactional records and media-object references intact.

12. Backend-first implementation phases with hard gates

Phase	Implementation scope	Exit gate
Phase 1 - Architecture, scope and contracts	Freeze user flows, resource booking types, permission matrix, API schemas, ERD and payment settlement assumptions.	Approved PRD, OpenAPI skeleton, ERD, ADRs; unresolved financial decisions recorded.
Phase 2 - Repository, Docker and infrastructure	Prepare Express modular monolith, worker process, Docker Compose, PostGIS, Redis, Nginx, object storage config, lint/tests.	Clean Docker-only boot, healthchecks and reproducible image build.
Phase 3 - Core middleware and migrations	Env schema, logging, errors, request validation, DB transaction helper, migrations, seeds, basic observability.	Migrations reproducible and integration tests pass on clean DB.
Phase 4 - Auth and authorization	JWT login/refresh/revocation, service token, users/RBAC, partner ownership primitives, rate limiting.	Identity, negative permission and cookie/BFF contract tests pass.
Phase 5 - Tourism directory and CMS	Regions, districts, categories, destination detail/gallery/FAQs, media, CMS, nearby discovery and search.	Published public resources searchable and complete from APIs only.
Phase 6 - Partner supply systems	Partner verification, hotels/rooms/rates, guide service schedule, experience sessions, package departures.	Owner/approval checks and all availability APIs pass.
Phase 7 - Booking engine	Quote calculation, booking snapshots, holds, expiry, request-to-book, cancellation rules and concurrency safeguards.	Simultaneous inventory tests prove no oversell.
Phase 8 - Payments and asynchronous jobs	Gateway implementation, signed webhooks, refund processing, outbox, BullMQ notification/voucher pipelines.	Payment replay, reconciliation and recovery tests pass.
Phase 9 - Account/admin/reporting	Trips, favorites, reviews, moderation, operational dashboard, permissioned	All role-specific workflows covered through

Phase	Implementation scope	Exit gate
	reporting and settings.	APIs.
Phase 10 - Production readiness & handover	Index tuning, security scan, data backup/restore, capacity tests, deployment, API documentation examples.	Final checklist signed off; frontend integration contract frozen.

12.1 Module-level completion rule

Do not call a backend module complete unless it has approved migration(s), entities/constraints, input schemas, policies, service logic, HTTP routes, error responses, audit/event behavior where needed, OpenAPI definitions, tests, and working Docker-based execution. Implement one module at a time without changing unrelated modules except explicit integration contracts.

13. Operational risks and decision register

Priority	Risk	Mitigation / decision
Critical	Partner settlement / merchant-of-record	Payment operations cannot enter production until money-flow ownership, receipts, taxation and payout scope are agreed.
Critical	Overselling inventory	Use PostgreSQL locking/constraints and concurrency tests; never trust Redis availability counters as authoritative.
Critical	Service token exposure	Keep token only in server runtime; no client env; rotate and bind to network/per-environment.
High	Booking hold vs late payment	Document expiry/reconciliation policy and handle payment after expiry without forced overbooking.
High	Guide schedule overlap	Use DB-backed range constraints or locks plus concurrency testing.
High	CMS injection	Schema allowlist; sanitize rich text; no executable section payloads.
High	Partner impersonation	Every write checks organization membership and owned resource, even when JWT role exists.
Medium	Missing Jharkhand content	Seed approved districts/categories and curate source-verified local attraction data.
Medium	Media file privacy	Public/private bucket paths, signed URLs, verification document retention.
Medium	Docker volume/data loss	Backups and restore drills; pin images; avoid destructive down commands.

14. Final backend acceptance and frontend handoff

[ ] All REST endpoints in section 5 are live, have OpenAPI contracts and respond correctly under permitted/denied access.

[ ] All database tables in section 4 exist with validated foreign keys, check constraints, indexes and migration rollback strategy.

[ ] Tourist registration/login/refresh/logout, service-token validation and every role ownership flow are tested.

[ ] Public attraction, district, geo search, hotel, guide, experience, package and CMS read APIs support frontend needs.

[ ] Partners can on-board and maintain all supply types entirely through APIs; admin can approve and moderate.

[ ] Booking quote/hold/approval/confirmation/cancel and all listed inventory models pass concurrent tests.

[ ] Gateway verification/refund/signature/replay protection and reconciliations work in sandbox and fault scenarios.

[ ] Redis queues, outbox, worker retries, vouchers and notification state operate across restart/redeploy.

[ ] Operational logs, structured errors, audit trails, health checks and reports are configured.

[ ] New development environment starts with Docker Compose; no host Node/Postgres/Redis installation.

[ ] Durable volumes/object storage and documented backups/restoration are proven.

[ ] Sanitized seed data and Postman/OpenAPI collection enable frontend developers to work without backend assumptions.

14.1 Frontend handoff package

- OpenAPI JSON/YAML plus hosted Swagger UI for all implemented endpoints.
- Documented BFF service-token header contract, JWT cookie handling, session refresh and logout semantics.
- Response/error schema definitions, enums, pagination filters, media URL policy and sample payloads.
- Seed identities for tourist, partner and admin with nonproduction demo data (not production secrets).
- Flowcharts and transition matrices for booking, payment, cancellation, refund and partner approval.
- Docker Compose development instructions, environment template, test command inventory and change log.

15. Final recommendation

Build and validate the complete backend before front-end work, but do not postpone API-contract review: contract-first design is necessary to prevent later Next.js/BFF mismatch. Start by freezing permission and inventory models, then Docker infrastructure, PostgreSQL migrations, JWT + static BFF service authentication, followed by directory/CMS, partner supply, bookings/payments and production validation.

The proposal intentionally separates core marketplace features from speculative extras. Its schema and endpoints are complete for the stated business scope, but production readiness also depends on gateway/settlement/legal decisions, real inventory policy, selected storage provider and verified tourism content.

Document status: Backend architecture and implementation specification - suitable as a development work plan and acceptance reference; final executable DDL and OpenAPI specifications must be generated during implementation.